Assignment: K-Means Clustering Implementation

Dataset Options (Choose One)

You must train your clustering model on a standard dataset and then test with your own custom data. For Example:

1. Customer Segmentation: Train on a standard Mall Customers dataset (Age, Annual Income, Spending Score).

- **Your Custom Data:** Survey 10 individuals from the real world. Record their Age, estimated Annual Income, and a self-rated Spending Score (1-100).

2. Morphological Measurement (e.g., Iris or Palmer Penguins): Train on a standard botanical or biological dataset.

- **Your Custom Data:** Go outside and physically collect 10 distinct leaves or flowers. Measure their length and width in centimeters using a ruler.

3. Image Color Quantization: Train on a standard high-resolution image to cluster its RGB pixel values into a reduced color palette.

- **Your Custom Data:** Take a photo with your smartphone. Use your trained model to compress the colors of your custom photo.

1. Build a Full Clustering Workflow

You must construct a comprehensive Machine Learning pipeline using Python and Scikit-Learn. Your pipeline must bridge standard dataset training with real-world testing.

- **Project on GitHub:** Storing code, custom data, and the trained model object (e.g., .pkl or .joblib).
- **Automatic data retrieval:** The notebook must automatically pull the standard dataset and your custom data directly from your GitHub repository.
- **Data Preprocessing:** Use sklearn.preprocessing for standardizing and scaling data.
- **Real-World Testing:** You must properly format your custom data, pass it through the pipeline, and use your trained model to assign clusters.

2. Perform Essential Data Preprocessing & Custom Data Creation

Distance-based clustering requires strict data formatting. Your pipeline must include:

Standard Dataset Preprocessing:

- **Transforms:** You must define a scaling pipeline. K-Means relies heavily on Euclidean distance; therefore, unscaled data will result in a penalty.

Custom Data Creation (The "Real-World" Task):

1. Collect your 10 distinct data points matching your chosen dataset's features.
2. Format them cleanly into a .csv file (or save as an image file if performing color quantization).
3. **Important:** Process this custom data to match the training data format exactly (identical column names, identical data types).

3. GitHub Repository Setup (Crucial Step)

You must host your project on GitHub to ensure reproducibility.

- **Folder Structure:**
  - dataset/: Upload your custom .csv or smartphone image here.
  - model/: After training, save your model file (e.g., 190110.pkl) and upload it here.
  - ID.ipynb: Your Google Colab notebook.
  - README.md: Display your final visual plots and a description of your clusters.
- **Requirement:** I must be able to view the repo and see your custom data and the saved model file.

4. Model Implementation in Google Colab

The notebook must demonstrate the following steps without requiring manual file uploads during runtime:

A. Data Loading (Automation)

- Use !git clone [your-repo-link] inside the notebook to download your custom data.
- Load the standard training data.

B. Optimal K Selection

- **The Elbow Method:** Write a loop to fit K-Means models for varying numbers of clusters.
- Calculate and store the Within-Cluster Sum of Squares (WCSS/Inertia) for each iteration.

C. Model Fitting

- Initialize KMeans with your optimal chosen value for K.
- Fit the model to your scaled standard dataset.
- **Saving:** Save the fitted model using joblib or pickle.

D. Real-World Prediction

- Load your custom data using pandas.
- **Crucial:** Apply the *exact same fitted scaler* used on the standard dataset to your custom data. Do not instantiate or fit a new scaler.
- Use model.predict() to assign your custom data points to the established clusters.

5. Include the Following Visuals

Generate and display the following within the notebook:

- **The Elbow Curve Plot:** A line graph showing the Number of Clusters (K) on the X-axis and WCSS on the Y-axis.
- **Cluster Scatter Plot:** A 2D or 3D scatter plot of your standard dataset showing the distinct clusters in different colors. You must plot the final Centroids (marked with a distinct symbol) on this graph.
- **Custom Prediction Output:** A formatted table or DataFrame displaying your 10 custom items alongside their assigned Cluster ID.
- **Cluster Interpretation:** A text cell containing a 2-3 sentence profile analyzing what your distinct clusters represent based on their centroids (e.g., "Cluster 0 represents high-income, low-spending subjects").

6. Submission

- **GitHub Link:** The URL to your public repository containing the custom data, the saved model, and the .ipynb file.
- **Colab Link:** The direct link to your notebook.
- **Constraint:** I must be able to click the Colab link, select "Run All", and the notebook should:
    1. Clone your repo.
    2. Determine the optimal K and fit the model.
    3. Process your custom data.
    4. Output the cluster assignments and visual plots automatically.

Penalty: Failing to maintain any of these automation instructions will result in a penalty. If I have to manually upload files or debug file paths to make your code run, points will be deducted.